Reports and recordings

This is the reference for everything a run produces: the **standard** reports you use every day, the **regulated** evidence pack you build when a quality team needs proof, and the screenshots, videos and traces behind both. If you only read one table, read the next one.

Standard or regulated?

	Standard report	**Regulated evidence pack**
For	developers, everyday runs, finding why a test failed	quality teams, auditors, client reviewers
This is the default	**yes**: every `npm test` and every CI run	no: you ask for it
Build it with	`npm test`, then `npm run report:allure`	`npm run test:capture`, then `npm run report:validation`
Where it ends up	`reports/standard/`	`reports/regulated/`
Start with	the Allure report, or Playwright's report	`validation-report.html` (one page, double-click)
Contains	Allure report, Playwright report	one validation report (+ PDF), traceability matrix, its own Allure report, raw results, every recording, manifest, checksums
Recordings	screenshot and video of failed tests only	screenshot, video and trace of every test, each with a timestamp bar
Requirement links	tags exist, nothing is built from them	a requirement-to-test matrix with a verdict per requirement
Integrity	none	a SHA-256 checksum for every file
In CI	built on every run, published to GitHub Pages	only when you start the workflow by hand and tick **regulated pack**
Overwritten	by the next run	by the next validation run, so copy an approved pack somewhere safe
Size and time	small and quick	large and slower (see "What it costs")

The same tests feed both. Nothing about a test changes between them.

Where everything is

All generated output is under `reports/`. Only `reports/README.md` is committed.

```
reports/
  _raw/          what the run wrote: results, screenshots, videos, traces. Overwritten every
run. Ignore it.
  standard/
    allure/        the Allure report                    npm run report:allure
    playwright/    Playwright's own report              npm run report
  regulated/     the evidence pack                       npm run report:validation
    validation-report.html   <- start here
    READ-ME-FIRST.txt, validation-report.pdf, traceability/, allure/, allure-results/,
junit.xml,
    manifest.json, SHA256SUMS
```

`npm run clean` empties `_raw`, `standard` and `regulated`. It leaves `reports/archive` alone, because sealed archives are audit records.

Screenshots, videos and traces

What you get by default

The defaults are Playwright's usual economical ones, set in `playwright.config.ts`:

Recording	Default	Meaning
Screenshot	`only-on-failure`	one picture of the page when a test fails
Video	`retain-on-failure`	recorded for every UI test, but kept only if the test fails
Trace	`on-first-retry`	recorded when a failed test is retried. CI retries twice, a local run does not

So a fully green run has no screenshots, videos or traces. That is expected, not a fault. API-only tests open no browser, so they never have a screenshot or video, only a trace when one is switched on.

What "for every step" means

- **The video** covers the whole test, so every step is in it, from the first navigation to the last check.
- **The trace** records every action (each click, fill, navigation, check) with a snapshot of the page before and after, the network calls and the console. Open it with `npx playwright show-trace <trace.zip>`, or from the Allure or Playwright report. This is the step-by-step view.
- **The screenshot** is one picture, taken when the test ends.

Playwright does not take a separate screenshot after each named step. If your quality team needs a picture per step in the report itself, that would be a new feature on top of this; say so and it can be added.

How to switch recording on

You want	Do this
Everything recorded for every test, locally	`npm run test:capture`
The same for some tests only	`npm run test:capture -- --grep @smoke` or `npm run test:capture -- tests/ui/checkout`
The same in CI	Actions, **QA Accelerator CI**, **Run workflow**, tick **Record a screenshot, video and trace for every test**
Only the timestamp bar, in a normal run	`EVIDENCE_STAMP=true npm test` (PowerShell: `$env:EVIDENCE_STAMP='true'; npm test`)
Traces only	`npx playwright test --trace on`
One spec file or one `describe` always recorded	add `test.use({ screenshot: 'on', video: 'on', trace: 'on' });` at the top of it
Everything, always (not recommended)	change `trace`, `screenshot` and `video` to `'on'` in `playwright.config.ts`

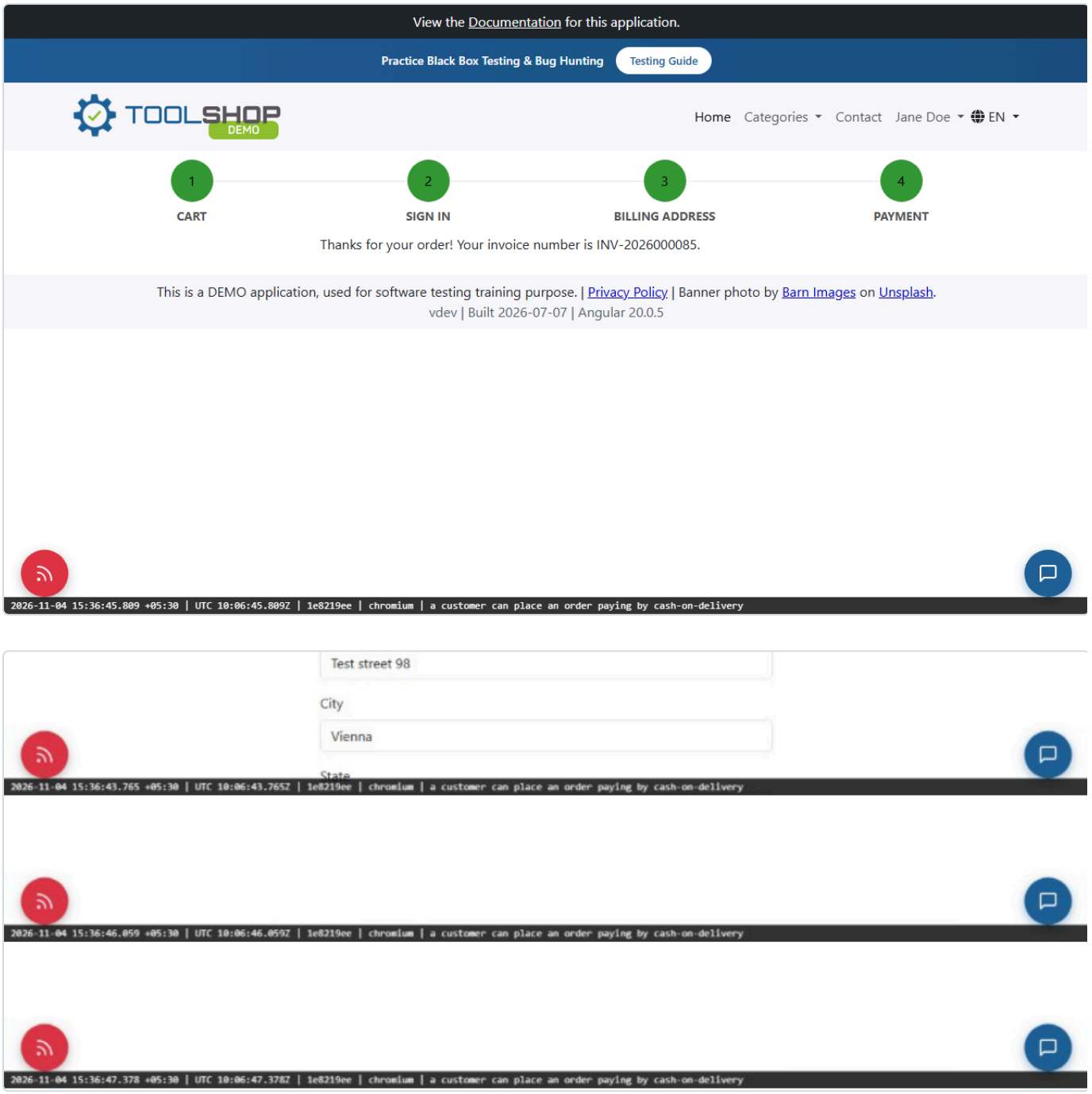
`npm run test:capture` uses `playwright.capture.config.ts`, which keeps the same suite and settings and changes three things: it records everything, it turns the timestamp bar on, and it sends Playwright's HTML report to `reports/_raw` so it never overwrites the standard one. It also clears `reports/_raw` first, so a validation run cannot contain results from an earlier run.

To go back to the defaults, just run `npm test`. Nothing needs resetting.

The timestamp bar

In capture mode (or with `EVIDENCE_STAMP=true`) every page shows a thin bar along the bottom with a live clock, so any screenshot or video frame shows when it was taken:

```
2026-11-04 14:34:15.341 +05:30 | UTC 09:04:15.341Z | 1e8219ee | chromium | a customer can place an order ...
```



It shows local time with milliseconds and the offset, the same moment in UTC, the commit, the browser and the test name. It lives in a closed shadow root and ignores the pointer, so locators and clicks are unaffected. It covers about 20 px at the bottom of the page. The time is the clock of the machine running the browser, so use a synchronised clock for formal runs. The code is in `src/utils/evidence-stamp.ts`.

The standard report

```
npm test                    # run (results go to reports/_raw)
npm run report:allure       # build reports/standard/allure and open it
npm run report              # open Playwright's report (traces, videos, screenshots of failed
tests)
```

- **Allure** shows the tests grouped by area, each with its steps, tags (including requirement IDs such as `REQ-AUTH-001`, which you can filter on), history and, for failed tests, the attachments.
- In CI it is built on every run, kept as an artifact for 14 days and, if you switched it on, published to GitHub Pages with trend history.
- `npm run report:allure:generate` builds it without opening it, and `npm run report:allure:open` opens an existing one.

The regulated evidence pack

Build one when a client or quality team needs proof of a run: which requirements were tested, what happened, and that the evidence was not changed afterwards.

```
npm run clean               # optional, but starts from nothing
npm run test:capture        # the whole suite, everything recorded and timestamped
npm run report:validation   # builds reports/regulated, with a PDF of the main report
```

- Use `npm run evidence` instead of `report:validation` for the same pack without the PDF (the PDF needs Chromium installed).
- To open the pack's Allure report, run `npm run report:validation:open`. It needs a small local server, so it can't be opened by double-clicking. `validation-report.html` can.
- To check that nothing changed, run `sha256sum -c SHA256SUMS` inside `reports/regulated`. Every line must say OK.
- To fail a pipeline when a requirement is failing or untested, run `node tools/traceability.mjs --strict`.

What is in the pack

File or folder	What it is
`validation-report.html`	**start here**: overall result, run details, traceability, known defects, gaps, all results, failures
`validation-report.pdf`	the same page as a PDF
`READ-ME-FIRST.txt`	a short guide to the folder, for whoever receives it

File or folder	What it is
`traceability/`	the requirement-to-test matrix as HTML, CSV and Markdown
`allure/`	the Allure report of the same run, titled "Validation Run", with no trend history
`allure-results/`	raw results, including every screenshot, video and trace
`junit.xml`	machine-readable results
`manifest.json`	commit, branch, who ran it, CI run, Node, Playwright and OS versions, lock-file hash, image digests
`SHA256SUMS`	a checksum for every file above

How requirements are linked

Each test names the requirement it verifies with a tag:

```
test('a customer can sign in', { tag: [Tags.smoke, ...requirement('REQ-AUTH-001')] }, ...)
test.describe('Checkout', { tag: [...requirement('REQ-ORD-001', 'REQ-ORD-003')] }, () => {
... })
```

The IDs and titles live in `docs/regulated/requirements.json` (the ones in the repository are samples for the demo application). A requirement gets one verdict: **Verified**, **Verified (known defect)**, **Failed**, **Not tested** or **Skipped**. A test written as an expected failure (`@known-issue`) marks its requirement as having a known defect, shown in amber, because the requirement is not fully met until the defect is fixed. `--grep @REQ-ORD-001` runs the tests of one requirement.

In CI

The regulated pack is not part of everyday runs. To build one on GitHub: Actions, **QA Accelerator CI**, **Run workflow**, then tick **regulated pack**, and tick **Record a screenshot, video and trace for every test** as well for a full validation run. The pack appears as the `evidence-<browser>` artifact. Artifacts expire, so copy an approved pack into the client's document system or immutable storage. The retention can be raised with the repository variable `EVIDENCE_RETENTION_DAYS` , within what your GitHub plan allows.

Dry run and formal run (regulated mode)

For a team that works with a dry run followed by a formal run, `npm run regulated:run` does the whole job: it labels the run, runs the suite in capture mode, builds the evidence pack and seals it as a zip with a checksum in `reports/archive` . A formal run is compared with its accepted dry run first. It is off until `config/regulated.json` has `"enabled": true` . The process, the settings and the CI workflow are in regulated/dry-run-and-formal-run.md.

```
npm run regulated:status                                    what is switched on
npm run regulated:run -- --type dry --build 2.4.1           rehearsal against the final build
npm run regulated:run -- --type formal --build 2.4.1 --against <dry run zip>
npm run regulated:verify -- <zip | folder | audit-log.jsonl>   check an archive later
npm run regulated:compare -- <dry run> <formal run>         compare two runs by hand
npm run regulated:note -- "what I did by hand, and why"     a note in the run's report
```

The sealed archives are in `reports/archive/` (git-ignored, like the rest of `reports/`). Copy an approved formal archive, with its `.sha256` file, into your records.

What it costs

Measured on this suite (89 tests, Chromium and the phone-sized project), on one Windows machine. Yours will differ.

	Normal run	**Capture run**
Time	about 2.5 minutes	about 6 minutes
Raw results	a few MB	about 225 MB (about 680 MB in `_raw` with Playwright's own report)
Regulated pack	not built	about 460 MB

This is why capture mode is kept for formal runs and demos, and `npm run clean` is there to free the space.

When something doesn't look right

- **"No videos for my passing tests."** Expected: they are kept only on failure. Use `npm run test:capture`.
- **A test shows two videos.** Playwright records one video per page. The accessibility scanner opens a second, blank page, so those tests have a tiny second video. The report numbers them and shows their sizes. Open the larger one.
- **The Allure build fails with `EPERM ... rename` on Windows.** A short-lived file lock after many large files were written. Run the command again. The evidence pack builds the report first and retries on its own.
- **`EBUSY` or "resource busy" when building the pack.** A file in `reports/regulated`, usually the PDF, is open in a viewer. Close it and run the command again.
- **The pack mentions tests I didn't just run.** The raw results are from an earlier run. Run `npm run clean`, or use `npm run test:capture`, which clears them itself.
- **A requirement shows "Not tested".** No test carries its ID. Add the tag, or accept the gap and record why.